

# Efficient Multi-instance Vector Commitment and Application to Post-quantum Signatures

Dung Bui

IRIF, Université Paris Cité, Paris, France  
bui@irif.fr

**Abstract.** The MPC-in-the-Head (MPCitH) and the VOLE-in-the-Head (VOLEitH) paradigms have recently been utilized to develop post-quantum signatures. Both rely on a mechanism that allows the signer to commit to  $N$  values and then later open all-but-one. In particular, MPCitH-based signatures achieve this using a puncturable pseudorandom function (PPRF) primitive, while VOLEitH-based signatures utilize an all-but-one vector commitment scheme.

A novel and efficient multi-instance PPRF, introduced by Bui *et al.* (Asiacrypt’24), provides a significant performance boost for MPCitH-based signatures, employing only a fixed-key block cipher to instantiate the PPRF while being provably secure in the ideal cipher model. This work presents an efficient multi-instance vector commitment derived from multi-instance PPRF. Our vector commitment scheme is secure in the multi-instance setting, when handling repetitive parallel executions. As a result, it can be directly applied to enhance the efficiency of VOLEitH-based signatures.

We implemented our vector commitment scheme into FAEST ([faest.info](https://github.com/dungbui/faest)), a round one candidate in the NIST post-quantum cryptography standardization. According to our experimental implementation, we achieve 10% – 27% improvement in both signing and verification times for various settings.

On the other hand, we adapt the VOLE-in-the-Head framework to the Multivariate Quadratic (MQ) problem, achieving a reduction in signature size. Specifically, for a 128-bit security level, our approach yields signature sizes between 3.1KB and 3.6KB with a small field size parameter.<sup>1</sup>

## 1 Introduction

**Zero-Knowledge, signature schemes based on MPCitH and VOLEitH.** Zero-knowledge (ZK) proof allows a prover to convince a verifier of their knowledge of a witness for an NP statement without revealing any additional information. Zero-knowledge proof has numerous applications in cryptography, especially for designing efficient signature schemes by applying the Fiat-Shamir transform [FS87] to convert any public-coin zero-knowledge proof system into a signature scheme. Since the existence of attacks from quantum computers [Sho94], the main focus is to design the digital signature scheme based on assumptions believed to resist quantum computer attacks. The recent call by NIST to standardize post-quantum cryptographic primitives has catalyzed research into efficient post-quantum signature schemes, some of which are using two main paradigms that are MPC-in-the-head (MPCitH) [IKOS07] and later VOLE-in-the-head (VOLEitH) [BBD<sup>+</sup>23].

The MPC-in-the-head paradigm allows the prover to “simulate” an MPC protocol while virtual parties receive shares of the witness and a target function that validates the correctness of the witness. Subsequently, the prover commits to the view of all parties and upon request from the verifier opens a random subset of these views. The verifier checks the consistency of these views and verifies that the output corresponds to the correct witness. Soundness is guaranteed by the inability of a cheating prover to generate consistent views for all parties. The zero-knowledge property is established through the security of the MPC protocol against an honest-but-curious adversary, who only gains access to the views of a subset of corrupted parties. The usage of this paradigm leads to efficient signatures [FJR22, AGH<sup>+</sup>23, CCJ23, BCC<sup>+</sup>24] from various assumptions having signature sizes from 3KB-7KB.

Another paradigm utilized in constructing signature schemes is VOLE-in-the-Head [BBD<sup>+</sup>23], which typically offers higher efficiency than most MPCitH signatures. VOLEitH signatures are constructed from VOLE-based zero-knowledge proofs [WYKW21, BMRS21, YSWW21, BCC<sup>+</sup>23]. Since the beginning, these zero-knowledge proofs have relied on the existence of vector oblivious linear function evaluation (VOLE) correlations between the prover and the verifier [BCG<sup>+</sup>19], which can be

<sup>1</sup> This is the full version of a published paper [Bui25].

later used to define IT-MAC. Due to this prerequisite, VOLE-based protocols are unable to operate in the public-coin model until the introduction of the VOLEitH approach by Baum *et al.* [BBD<sup>+</sup>23]. In addition to a generic public-coin zero-knowledge proof system, Baum *et al.* employs this approach to develop the FAEST post-quantum signature scheme, solely based on AES and hash functions. Later, [CLY<sup>+</sup>24] introduces a signature scheme based on the regular syndrome decoding assumption with signature sizes of 4KB for the security parameter of 128 bits and it combines the VOLE-in-the-Head technique from [BBD<sup>+</sup>23] with a sketching method of [BGI16] to reduce the check of the noise structure to a system of degree-2 equations, which are then proven using the QuickSilver VOLE-based zero-knowledge proof [YSWW21].

**Puncturable PRF and All-but-one Vector Commitment.** The MPCitH and VOLEitH schemes both require committing to  $N$  random values and subsequently opening all-but-one of these values to ensure the soundness of the underlying ZK proof. To efficiently meet this requirement, MPCitH schemes utilize PPRF (puncturable pseudorandom function), while VOLEitH schemes employ all-but-one VC (vector commitment) to formalize the hiding and binding properties essential to the VOLEitH paradigm. Although both approaches are constructed using the length-doubling GGM tree [GGM84], their construction methodologies and security constraints differ.

Specifically, PPRF [KPTZ13, BW13, BGI14, GGM84] enables the prover to succinctly open  $N - 1$  views (the leaves of the tree) by revealing  $\log(N)$  co-path nodes of the tree, ensuring the hiding property (*i.e.*, given  $N - 1$  revealed values, the unopened secret remains indistinguishable from random). Conversely, VC [BBD<sup>+</sup>23] can be conceptualized as a tree expanded using PPRF, albeit with a distinct method of constructing the leaves (the last layer), such that both the hiding and binding properties are preserved. Consequently, by adjusting the construction method of the last layer in VC, we show that our VC can also fit within the MPCitH framework.

To achieve the desired level of security while maintaining the efficiency of signatures, repetition is unavoidable in both PPRF and VC. In this work, we explore the concept of multi-instance VC in a repetitive parallel setting and provide a concrete security analysis while maintaining efficiency. This is made possible by leveraging the multi-instance PPRF primitive [BCC<sup>+</sup>24]. We believe our multi-instance VC will be useful for VOLEitH signatures and also for many other MPCitH signatures.

**Code-based signature schemes from Multivariate Quadratic (MQ).** A multivariate quadratic map  $\mathcal{F} : \mathbb{F}_p^n \rightarrow \mathbb{F}_p^m$  is a system of  $m$  quadratic polynomials in  $n$  variables defined over some finite field  $\mathbb{F}_p$ . The  $\text{MQ}_{p,m,n}$  problem is a code-based assumption such that for uniformly random  $\mathcal{F} : \mathbb{F}_p^n \rightarrow \mathbb{F}_p^m$  and  $\mathbf{x} \in \mathbb{F}_p^n$ , with a suitable parameters set, finding  $\mathbf{x}$  given  $\mathcal{F}$  and  $\mathcal{F}(\mathbf{x})$  is hard. The average-case hardness of the MQ is one of the leading candidate post-quantum cryptographic assumptions. The Rainbow signature scheme [DS05], stands out as one of the earliest and most studied signature schemes in multivariate cryptography. Nevertheless, its security has been reduced by recent attacks [Beu21, Beu22]. While there existed efficient signatures [FJR22, AGH<sup>+</sup>23, CCJ23, BCC<sup>+</sup>24] based on concrete code-based assumption as syndrome decoding having the signature size of less than 5KB, signatures based on MQ assumption [Beu20, Wan22, Fen22] are less efficient, *e.g.*, have a larger signature size ranging from 7KB-14KB. This leads to the question of whether we can construct an efficiently secure signature scheme based on the multivariate quadratic problem with a *smaller signature size*. Our second contribution is to get an MQ-based signature scheme by considering the hardness assumption in the VOLEitH paradigm as the MQ problem.

### 1.1 Our contribution

We introduce a novel multi-instance vector commitment scheme designed to optimize any MPCitH-based and VOLEitH-based signatures by directly integrating them with our proposed vector commitment.

- Our vector commitment is based on multi-instance PPRF [BCC<sup>+</sup>24] and its performance outperforms the GGM-tree-based VC proposed in [BBD<sup>+</sup>23].
- We give detailed proofs to show that our VC not only enhances performance but also maintains security properties in the random-oracle model, including multi-hiding and extractable binding.

- Since our VC is defined in a multi-instance setting, where repetition and signature queries are inherently integrated into the security proofs, therefore, it directly can replace the combination of hash-based PPRF and commitments in MPCitH-based signatures (under some constraints) as well as VC in VOLEitH-based signatures in the repetitive setting.

For concrete efficiency, with a security level of 128 bits, our GGM-tree vector commitment is instantiated by using the AES fixed-key to expand the tree and SHAKE128 to compute the commitments. When integrating our vector commitment into the FAEST signature in various setting, our vector commitment gains from 10% to 27% improvement.

## 1.2 Related Work

Recently, Bui *et al.* [BCdSG24] also proposed a vector commitment construction based on the half-tree technique. This construction is efficient as it reduces the number of AES calls to expand the tree by half. Their construction can be integrated into our multi-instance vector commitment, but we would have to adapt our analysis to the ideal cipher model to get provable security in a multi-instance setting. Since half tree techniques have two properties: 1) the sum of all nodes on every layer of the tree is always equal to a certain offset  $\Delta$ , and 2) AES fixed-key also used to instantiate the PPRF while the number of AES used is half compared to standard GGM-tree based PPRF, we believe that security loss of half-tree-based PPRF in the multi-instance setting is larger than our construction.

Following the footprint of VOLEitH-based signature, recently, [BBM<sup>+</sup>24a] proposes optimizations for VOLEitH-based signatures that include 1) new construction of batch vector commitment and 2) new VOLEitH-based signatures from various hardness assumptions, one of them (called KuMQuat) is also based on MQ problem. KuMQuat has the smallest signature size compared to other VOLEitH signatures (ranging from 2555B to 3588B for the security level I), and [BBM<sup>+</sup>24a] provided a benchmark for showing the performance of KuMQuat is still competitive.

## 2 Technical Overview

### 2.1 Vector commitment from multi-instance PPRF

Starting point from a GGM-tree-based PPRF, where prover has a master key *seed* and then uses length-doubling PRG to expand this seed into a binary tree of width  $2^D$ , having  $2^D$  leaves  $\{\text{seed}_1, \dots, \text{seed}_{2^D}\}$ . Later, to open all leaves except the leaf at index  $i \in [2^D]$ , prover only needs to reveal  $D$  co-path node values related to the  $i$ -th leaf, which allows verifier to learn  $(\text{seed}_j)_{j \in [2^D] \setminus \{i\}}$  by using PRG to reconstruct the tree. Therefore, the communication cost to open all but one is succinctly reduced from  $O(2^D)$  bits to  $O(D)$  bits. Multi-instance PPRF [BCC<sup>+</sup>24], which is a generalization of PPRF, is introduced to capture the setting where  $N$  instances of  $\tau$  repetitions of a PPRF are used (Definition 6).

From multi-instance PPRF, we show how to construct multi-instance vector commitment that has stronger security constraints, the hiding (Definition 7), and binding properties (Definition 8). In our construction, there are three main steps:

- First, we use multi-instance PPRF to expand a master key *seed* into a binary tree having  $2^D$  leaves  $(\text{seed}_j)_{j \in [2^D]}$  by using a *salt* to determine the permutations inside PRG. The usage of multi-instance PPRF contributes to the proof of hiding property.
- From each  $(\text{seed}_j)_{j \in [2^D]}$ , we apply a hash function  $H$  on it to get a pair of  $(\text{sd}_j, \text{com}_j) \in \{0, 1\}^\lambda \times \{0, 1\}^{2\lambda}$ . To prove security, the hash function is later modeled as a random oracle. The commitment  $\text{com}_j$  allows prover to commit the corresponding  $\text{seed}_j$  and to not cheat in binding games.
- To optimize the communication cost as in [BBD<sup>+</sup>23], we use another random oracle to hash all  $(\text{com}_j)_{j \in [2^D]}$  and get a global commitment  $\text{com}$ .

Finally, we adapt all steps to the repetitive setting to get  $(N, \tau)$ -instance vector commitment. For opening all-but-one  $i$ -th leaf, prover reveals the opening for multi-instance PPRF together with  $(\text{com}_i, \text{com})$ . Verifier then can self-compute every leaf of the PPRF-tree except the  $i$ -th leaf, from that verifier reconstructs  $(\text{sd}_j, \text{com}_j)_{j \neq i}$ . Because verifier also gets  $(\text{com}_i, \text{com})$  so verifier can verify the validity of commitment and it allows the fact that the prover can only cheat on hidden leaf  $\text{seed}_i$  with negligible probability once it is committed.

For security, we prove in a hybrid game, the hiding is followed directly by the randomness of multi-instance PPRF and correlated resistance of the hash function (used in the last layer). For the binding game, we highlight that since the random oracle is used to get  $(\text{sd}_j, \text{com}_j)_{j \in [2^D]}$  where the output length of  $\text{com}_j \in \{0, 1\}^{2\lambda}$ , the advantage of adversary intuitively is bounded by the collision probability of commitments  $O(2^{-2\lambda})$ .

## 2.2 Faster MPCitH and VOLEitH-based signatures

We show how our vector commitment scheme integrates into the signature construction. Both MPCitH-based and VOLEitH-based signatures are developed in repetitive parallel settings, though they differ in how our vector commitment is applied.

For VOLEitH-based signatures, the leaves of the GGM tree are used to establish the VOLE correlation using the SoftspokenOT technique [Roy22]. In this context, our vector commitment scheme perfectly fits into the signature construction. On the other hand, for MPCitH-based signatures, the output domain of the random oracle  $\mathbf{H}$  needs to be adjusted depending on specific signature construction.

In particular, in the case of MPCitH-based signatures, as demonstrated in [BCC<sup>+</sup>24], a PPRF can be directly applied to the signature scheme to obtain the leaves  $\text{seed}_i$ . The prover then uses these leaves as inputs to a PRG to generate randomness and commitment values. The generated randomness later is added to witness shares to hire the actual witness value, while the commitment value ensures that the prover cannot cheat. To prove the soundness of the underlying identification scheme, the PRG is typically modeled as a random oracle [FJR22, AGH<sup>+</sup>23, CCJ23, BCC<sup>+</sup>24]. This approach aligns well with our vector commitment scheme, where the last layer is instantiated by a random oracle. Thus, our vector commitment can be integrated into MPCitH-based signatures by simply modifying the output domain of the random oracle.

## 3 Preliminaries

### 3.1 Notation

Given a set  $S$ , we write  $s \leftarrow_r S$  to indicate that  $s$  is uniformly sampled from  $S$ . Given a probabilistic Turing machine  $\mathcal{A}$  and an input  $x$ , we write  $y \leftarrow_r \mathcal{A}(x)$  to indicate that  $y$  is sampled by running  $\mathcal{A}$  on  $x$  with a uniform random tape, or  $y \leftarrow \mathcal{A}(x; r)$  when we want to make the random coins explicit. Given an integer  $n \in \mathbb{N}$ , we denote by  $[n]$  the set  $\{1, \dots, n\}$  and  $[0, n)$  for the set  $\{0, \dots, n-1\}$ . We use  $\lambda$  for the computational security parameter. We let  $\text{negl}(\lambda)$  denote any function that is negligible in the security parameter.

### 3.2 Cryptographic Definitions

**Basic definitions.** We define the definitions of secure PRG and collision-resistant hash function as below.

**Definition 1 (Indistinguishability).** Two distributions  $X, Y$  are defined as  $(t, \epsilon)$ -indistinguishable if for an algorithm  $D : \{0, 1\}^m \rightarrow \{0, 1\}$  running in time  $t$ , we have  $|\Pr[D(X) = 1] - \Pr[D(Y) = 1]| \leq \epsilon$ .

**Definition 2 ( $(t, \epsilon_{\text{PRG}^m})$ -secure PRG).** Let  $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{l(\lambda)}$  where  $l(\lambda)$  be a polynomial in  $\lambda$ . Then,  $G$  is a  $(t, \epsilon_{\text{PRG}^m})$ -secure pseudorandom generator if

- Expansion:  $l(\lambda) > \lambda$ ;
- For any PPT  $\mathcal{A}$  running in time  $t$ , the distributions  $\{G(s) | s \leftarrow \{0, 1\}^\lambda\}$  and  $\{r | r \leftarrow \{0, 1\}^{l(\lambda)}\}$  are  $\epsilon_{\text{PRG}^m}$ -indistinguishable.

The generalization of  $(t, \epsilon_{\text{PRG}^m})$ -secure PRG is introduced in [BCC<sup>+</sup>24] as  $(N, \tau)$ -instance  $(t, \epsilon)$ -secure PRG. The length-doubling PRG in [BCC<sup>+</sup>24] is built from two different functions  $(F_0, F_1)$ . In particular, given a PRG that takes a seed  $\text{seed} \in \{0, 1\}^\lambda$  and a random salt  $\text{salt} = (\text{salt}_0, \text{salt}_1) \in \{0, 1\}^{2\lambda}$  as inputs then for  $i \in \{0, 1\}$ , the inputs of  $F_i$  are  $\text{seed}$  and additionally salt  $\text{salt}_i$ , i.e.,  $\text{PRG}(\text{seed}, \text{salt}) \rightarrow (F_0(\text{seed}, \text{salt}_0), F_1(\text{seed}, \text{salt}_1))$ . In the  $N$ -instance security game, the distinguisher  $\mathcal{D}$  tries to distinguish between random values and the set of  $(F_0(\text{seed}_i, \text{salt}_{i,0}), F_1(\text{seed}_i, \text{salt}_{i,1}))$  and this game is repeated  $\tau$  times while  $\text{salt}$  is fixed through all repetition.

**Definition 3 (( $(N, \tau)$ -instance  $(t, \epsilon)$ -secure PRG [BCC<sup>+</sup>24]).** A PRG  $\text{PRG} = (F_0, F_1)$  with  $F_b : \{0, 1\}^{2\lambda} \rightarrow \{0, 1\}^\lambda$  is an  $(N, \tau)$ -instance  $(t, \epsilon)$ -secure length-doubling PRG if for every non-uniform PPT distinguisher  $\mathcal{D}$  running in time at most  $t$ , it holds that for all sufficiently large  $\lambda$ ,

$$\text{Adv}^{\text{PRG}}(\mathcal{D}) = |\Pr[\text{Exp}_{\mathcal{D}}^{\text{rw-prg}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{D}}^{\text{iw-prg}}(\lambda) = 1]| \leq \epsilon(\lambda),$$

where  $\text{Exp}_{\mathcal{D}}^{\text{rw-prg}}(\lambda)$  and  $\text{Exp}_{\mathcal{D}}^{\text{iw-prg}}(\lambda)$  are defined below.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Exp}_{\mathcal{D}}^{\text{rw-prg}}(\lambda) :$<br><ul style="list-style-type: none"> <li>– <math>(\text{salt}_1, \dots, \text{salt}_{2N}) \leftarrow_r (\{0, 1\}^\lambda)^{2N}</math></li> <li>– <math>(\text{seed}_{i,e})_{i \leq N, e \leq \tau} \leftarrow_r (\{0, 1\}^\lambda)^{N \cdot \tau}</math></li> <li>– <math>\forall i \leq N, e \leq \tau :</math> <ul style="list-style-type: none"> <li>• <math>y_{2i-1,e} \leftarrow F_0(\text{seed}_{i,e}, \text{salt}_{2i-1})</math></li> <li>• <math>y_{2i,e} \leftarrow F_1(\text{seed}_{i,e}, \text{salt}_{2i})</math></li> </ul> </li> </ul> <p><b>Output</b> <math>b \leftarrow \mathcal{D}((\text{salt}_i, (y_{i,e})_{e \leq \tau})_{i \leq 2N})</math></p> | $\text{Exp}_{\mathcal{D}}^{\text{iw-prg}}(\lambda) :$<br><ul style="list-style-type: none"> <li>– <math>(\text{salt}_1, \dots, \text{salt}_{2N}) \leftarrow_r (\{0, 1\}^\lambda)^{2N}</math></li> <li>– <math>(y_{i,e})_{i \leq 2N, e \leq \tau} \leftarrow_r (\{0, 1\}^\lambda)^{2N \cdot \tau}</math></li> </ul> <p><b>Output</b> <math>b \leftarrow \mathcal{D}((\text{salt}_i, (y_{i,e})_{e \leq \tau})_{i \leq 2N})</math></p> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Definition 4 (Collision-Resistant Hash Functions).** A family of functions  $\text{Hash}_k : \{0, 1\}^* \rightarrow \{0, 1\}^{l(\lambda)}$ ;  $k \in \{0, 1\}^{\kappa(\lambda)}$  indexed by a security parameter  $\lambda$  is collision-resistant if there exists a negligible function  $v$  such that, for any PPT algorithm  $\mathcal{A}$ , we have:

$$\Pr \left[ \begin{array}{c} x \neq x' \\ \cap \text{Hash}_k(x) = \text{Hash}_k(x') \end{array} \middle| \begin{array}{c} k \in \{0, 1\}^{\kappa(\lambda)} \\ (x, x') \leftarrow \mathcal{A}(k) \end{array} \right] \leq v(\lambda)$$

**Lemma 5 (Collisions in random oracle).** Given a collision-resistant hash function  $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$  and  $H$  is modeled as a random oracle. For any PPT algorithm making  $q$  queries to  $H$  has a probability of at most  $q^2/2^{2\lambda}$  encountering a collision.

### 3.3 The MPC-in-the-Head Paradigm

The MPC-in-the-head paradigm was initiated by the work of Ishai *et al.* [IKOS07] and provided a compiler that can build honest-verifier zero-knowledge (HVZK) proofs for arbitrary circuits from secure MPC protocols. Assume we have an MPC protocol with the following properties:

- $N$  parties  $(P_1, \dots, P_N)$  securely and jointly evaluate a function  $f : \{0, 1\}^* \rightarrow \{0, 1\}$  on  $\mathbf{x}$  while each party possess an additive share  $\llbracket \mathbf{x} \rrbracket_i$  of input  $\mathbf{x}$ ,
- Secure against passive corruption of  $N - 1$  parties, *i.e.*, any  $(N - 1)$  parties can not recover any information about the secret  $\mathbf{x}$ .

Then the HVZK proof of knowledge of  $\mathbf{x}$  such that  $f(\mathbf{x}) = 1$  is constructed as:

- Prover generates the additively shares of the witness  $\mathbf{x}$  into  $(\llbracket \mathbf{x}_1 \rrbracket, \dots, \llbracket \mathbf{x}_N \rrbracket)$  among  $N$  virtual parties  $(P_1, \dots, P_N)$  and emulate the MPC protocol “in-the-head”.
- Prover commits to the view of each party and sends commitments to the verifier.
- Verifier chooses randomly  $(N - 1)$  parties and asks the prover to reveal the view of these parties except one. The verifier later accepts if all the views are consistent with an honest execution of MPC protocol with output 1 and agrees with the commitments.

Security of MPC protocol implies that the verifier learns nothing about the input  $\mathbf{x}$  from the  $N - 1$  shares, and MPC correctness guarantees that prover can only cheat with probability  $1/N$ . Security can then be amplified with parallel repetitions.

### 3.4 Multi-instance Puncturable PRF

**Puncturable PRF.** A puncturable pseudorandom function (PPRF) [BGI14] is a PRF  $F$  such that given an input  $x$ , and a PRF master key  $K$ , one can generate a *punctured* key, denoted  $K\{x\} = F.\text{Punc}(K, x)$ , which satisfies:

- There is an algorithm  $F.\text{Eval}$  such that  $F.\text{Eval}(K\{x\}, x') = F_K(x')$  for all  $x' \neq x$  (evaluating  $F$  at every point except for  $x$ ).
- $F_K(x)$  is indistinguishable from random given  $K\{x\}$ .

The concept of PPRF can be instantiated efficiently by using the length-doubling GGM tree [GGM84], for a node  $x$ , two children in the next level is computed from a PRG function defined as  $\text{PRG}(x) \rightarrow (F_0(x), F_1(x))$ . For a tree of  $N$  leaves, given a  $x \in [N]$ , the punctured key  $K\{x\}$  is the set of  $\log N$  co-path values that allows computing all leaves of the tree except the leave at  $x$ -th position while the PRF *master* key is the root of the tree. There are several ways to construct PRG [BCC<sup>+</sup>24, BCdSG24, GYW<sup>+</sup>23] with corresponding different security analyses, however, the common manner is using a fix-key block cipher to take advantage of the hardware acceleration for efficiency, for example, AES block cipher.

**Multi-instance PPRF.** The concept of multi-instance PPRF was introduced in [BCC<sup>+</sup>24] to generalize the construction of PPRFs with repetition parameter  $\tau$ . The primary motivation behind using  $(N, \tau)$ -multi-instance PPRF is to replace hash-based PPRFs in MPCitH signatures then speed up the performance. Unlike a single PPRF-based signature, where the security loss is proportional to the number of signature queries from an adversary, using multi-instance PPRF in signature construction results in a more tightly reduced unforgeability, directly tied to the security of the multi-instance PPRF itself.

We then recall the formal definition of  $\tau$ -multi-instance PRF proposed in [BCC<sup>+</sup>24]. Informally, given construction of  $N$ -instances of PPRF is defined by a set of  $N$  *master* keys  $(K_1, \dots, K_N)$  with an addition set of *salt* as  $(\text{salt}_1, \dots, \text{salt}_N)$ . Samples a set of  $N$  inputs  $(x_1, \dots, x_N)$  to PPRFs and gives the set of *punctured* keys  $(K_1\{x_1\}, \dots, K_N\{x_N\})$  to a distinguisher  $\mathcal{D}$ , then for any distinguisher  $\mathcal{D}$  running in time at most  $t$  then  $\mathcal{D}$  can distinguish between the real world and the ideal world experiments with a negligible probability  $\epsilon(\lambda)$ . Specifically, in the former experiment,  $\mathcal{D}$  gets set of evaluations  $(F_{K_1}(x_1), \dots, F_{K_N}(x_N))$  while in the latter one,  $\mathcal{D}$  gets a set of  $N$  random values. And with a repetition  $\tau$ , it is considered as  $(N, \tau)$ -instance  $(t, \epsilon)$ -secure PPRF if for any distinguisher  $\mathcal{D}$  running in time at most  $t$  then  $\mathcal{D}$  can not distinguish the game even it is repeated  $\tau$ -times under the condition of keeping the set of *salt* to be fixed.

**Definition 6** ( $(N, \tau)$ -instance  $(t, \epsilon)$ -secure PPRF [BCC<sup>+</sup>24]). *A function family  $F = \{F_K\}$  with input domain  $[2^D]$ , salt domain  $\{0, 1\}^s$ , and output domain  $\{0, 1\}^\lambda$ , is an  $(N, \tau)$ -instance  $(t, \epsilon)$ -secure PPRF if it is a PPRF which additionally takes as input a salt *salt*, and for every non-uniform PPT distinguisher  $\mathcal{D}$  running in time at most  $t$ , it holds that for all sufficiently large  $\lambda$ ,*

$$\text{Adv}^{\text{PPRF}}(\mathcal{D}) = |\Pr[\text{Exp}_{\mathcal{D}}^{\text{rw-pprf}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{D}}^{\text{iw-pprf}}(\lambda) = 1]| \leq \epsilon(\lambda)$$

where the experiments  $\text{Exp}_{\mathcal{D}}^{\text{rw-pprf}}(\lambda)$  and  $\text{Exp}_{\mathcal{D}}^{\text{iw-pprf}}(\lambda)$  are defined below.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{Exp}_{\mathcal{D}}^{\text{rw-pprf}}(\lambda) :$ <ul style="list-style-type: none"> <li>– <math>((K_{j,e})_{j \leq N, e \leq \tau} \leftarrow_r (\{0, 1\}^\lambda)^{N \cdot \tau}</math></li> <li>– <math>\text{salt} := (\text{salt}_1, \dots, \text{salt}_N) \leftarrow_r \{0, 1\}^s</math></li> <li>– <math>\mathbf{i} := ((i_{1,e})_{e \leq \tau}, \dots, (i_{N,e})_{e \leq \tau}) \leftarrow_r [2^D]^{N \cdot \tau}</math></li> <li>– <math>\forall j \leq N, e \leq \tau : K_{j,e}^{i_{j,e}} \leftarrow F.\text{Punc}(K_{j,e}, i_{j,e})</math></li> <li>– <math>(y_{j,e})_{j \leq N, e \leq \tau} \leftarrow (F_{K_{j,e}}(i_{j,e}, \text{salt}_i))_{j \leq N, e \leq \tau}</math></li> </ul> <p><b>Output</b> <math>b \leftarrow \mathcal{D}(\text{salt}, \mathbf{i}, (K_{j,e}^{i_{j,e}}, y_{j,e})_{j \leq N, e \leq \tau})</math></p> | $\text{Exp}_{\mathcal{D}}^{\text{iw-pprf}}(\lambda) :$ <ul style="list-style-type: none"> <li>– <math>((K_{j,e})_{j \leq N, e \leq \tau} \leftarrow_r (\{0, 1\}^\lambda)^{N \cdot \tau}</math></li> <li>– <math>\text{salt} := (\text{salt}_1, \dots, \text{salt}_N) \leftarrow_r \{0, 1\}^s</math></li> <li>– <math>\mathbf{i} := ((i_{1,e})_{e \leq \tau}, \dots, (i_{N,e})_{e \leq \tau}) \leftarrow_r [2^D]^{N \cdot \tau}</math></li> <li>– <math>\forall j \leq N, e \leq \tau : K_{j,e}^{i_{j,e}} \leftarrow F.\text{Punc}(K_{j,e}, i_{j,e})</math></li> <li>– <math>(y_{j,e})_{j \leq N, e \leq \tau} \leftarrow_r (\{0, 1\}^\lambda)^{N \cdot \tau}</math></li> </ul> <p><b>Output</b> <math>b \leftarrow \mathcal{D}(\text{salt}, \mathbf{i}, (K_{j,e}^{i_{j,e}}, y_{j,e})_{j \leq N, e \leq \tau})</math></p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3.5 Vector Commitment

A (non-interactive) VC scheme (with message space  $\mathcal{M}$  and commitment space  $\mathcal{C}$ ) is defined by four PPT algorithms (*Setup*, *Commit*, *Open*, *Verify*). We follow the security definitions of VC in [BBD<sup>+</sup>23], the hiding is used to prove that the adversary learns nothing from the VC.Open even if the adversary is allowed to choose the opening challenge and the extractable-binding game shows that after committing, an adversary is bound to the messages contained in the commitments, except for the one(s) that it does not open.

**Definition 7 (VC Hiding [BBD<sup>+</sup>23]).** Let  $\text{VC}$  be a vector commitment scheme in the  $\text{RO}$  model with random oracles  $\text{H}$  and  $\text{H}_1$ . The adaptive hiding experiment for  $\text{VC}$ ,  $\text{AdpHiding}$ , with  $N = \text{poly}(\lambda)$  and stateful  $\mathcal{A}$  is defined as follows.

1.  $\text{pp} \leftarrow \text{Setup}(1^\lambda, N)$
2.  $b \leftarrow \$ \{0, 1\}$
3.  $\text{seed} \leftarrow \$ \{0, 1\}^\lambda$
4.  $(\text{com}, \text{decom}, (\text{sd}_1^*, \dots, \text{sd}_N^*)) \leftarrow \text{Commit}(\text{seed})$
5.  $I \leftarrow \mathcal{A}(1^\lambda, \text{pp}, \text{com})$
6.  $\text{pdecom}_I \leftarrow \text{Open}(\text{decom}, I)$
7.  $\text{sd}_i \leftarrow \text{sd}_i^*$  for  $i \in I$
8. For  $i \notin I$ ,  $\text{sd}_i \leftarrow \begin{cases} \text{sd}_i^*, & \text{if } b = 0 \\ \leftarrow \$ \mathcal{M}, & \text{otherwise} \end{cases}$
9.  $b' \leftarrow \mathcal{A}(\{\text{sd}_i\}_{i \in [N]}, \text{decom}_I)$
10. Output 1 (win) if  $b' = b$  else 0 (lose).

The selective hiding experiment for  $\text{VC}$ ,  $\text{SelHiding}$ , is defined similarly but  $\mathcal{A}$  must choose  $I$  prior to receiving  $\text{com}$ .

We define  $\mathcal{A}$ 's advantage as:

$$\text{Adv}_{\mathcal{A}, \text{VC}}^{\text{AdpHiding}} = \Pr[\mathcal{A} \text{ wins } \text{AdpHiding}] - \frac{1}{2},$$

and similarly for the  $\text{SelHiding}$  game.

We say  $\text{VC}$  is adaptively (resp. selectively) hiding if every PPT adversary  $\mathcal{A}$  has a negligible advantage in the respective game.

**Definition 8 (VC Extractable-Binding [BBD<sup>+</sup>23]).** Let  $\text{VC}$  be a VC constructed from two random oracles  $\text{H}$  and  $\text{H}_1$ ,  $\mathcal{A}$  be any PPT adversary with oracle access to  $\text{H}$  and  $\text{H}_1$ , resulting in query-response lists  $Q$  and  $Q_1$  respectively.

An extractor  $\text{Ext}$  be a PPT algorithm defined as:

- $\text{Ext}(\text{pp}, (Q, Q_1), \text{com}) \rightarrow (\text{sd}_i)_{i \in [N]}$ : given an instance of  $\text{VC}$  public parameters  $\text{pp}$ , query-response lists  $Q$  and  $Q_1$ , and a commitment  $\text{com} \in \mathcal{C}$ , return the messages committed to by  $\text{com}$ . ( $\text{Ext}$  may output  $\text{sd}_i = \perp$  to signify an invalid commitment.)

For any  $N \leq \text{poly}(\lambda)$  and stateful adversary  $\mathcal{A}$ , we define the straight-line extractable/binding game,  $\text{ExtBind}$ , against  $\text{VC}$  as follows:

1.  $\text{pp} \leftarrow \text{Setup}^{\text{H}, \text{H}_1}(1^\lambda, N)$
2.  $\text{com} \leftarrow \mathcal{A}^{\text{H}, \text{H}_1}(1^\lambda, \text{pp})$
3.  $\{\text{sd}_i^*\}_{i \in [N]} \leftarrow \text{Ext}(\text{pp}, (Q, Q_1), \text{com})$
4.  $(I, \text{pdecom}_I) \leftarrow \mathcal{A}^{\text{H}, \text{H}_1}(\text{open})$
5. Output 0 (failure) if:
  - $\text{Verify}^{\text{H}, \text{H}_1}(\text{com}, j^*, \text{pdecom}_{j^*}) = \perp$ , i.e., the opening is not valid; or
  - $\text{Verify}^{\text{H}, \text{H}_1}(\text{com}, j^*, \text{pdecom}_{j^*}) = \{\text{sd}_i\}_{i \in I}$  and  $\text{sd}_i = \text{sd}_i^*$  for all  $i \neq j^*$ , i.e., the opening is valid but extraction by  $\text{Ext}$  was successful
6. Otherwise, output 1 (success).

We define  $\mathcal{A}$ 's advantage as  $\text{Adv}_{\mathcal{A}, \text{VC}}^{\text{ExtBinding}} = \Pr[1 \leftarrow \text{ExtBind}]$ , and say  $\text{VC}$  is straight-line extractable-binding w.r.t.  $\text{Ext}$  if

$$\text{Adv}_{\mathcal{A}, \text{VC}}^{\text{ExtBinding}} \leq \text{negl}.$$

### 3.6 Multivariate Quadratic Problem

Given a triple of parameters  $(p, m, n) \in \mathbb{N}$ , the Multivariate Quadratic Problem asks to find a vector solution in  $\mathbb{F}_p^n$  (assume it exists) of a random system of  $m$  linear equations over  $\mathbb{F}_p$ .

**Definition 9 (Multivariate Quadratic Problem - Matrix form).** Let  $\mathbb{F}_p$  be the finite field. Let  $(m, n)$  be positive integers. The multivariate quadratic problem  $\text{MQ}_{p, m, n}$  with parameters  $(p, m, n)$  is the following problem:

- (Problem generation) Sample  $\mathbf{x} \leftarrow_r \mathbb{F}_p^n$  and  $(\mathbf{A}_i)_{i \leq m} \leftarrow_r \mathbb{F}_p^{n \times n}$ ,  $(\mathbf{b}_i)_{i \leq m} \leftarrow_r \mathbb{F}_p^n$ .  
Set  $y_i \leftarrow \mathbf{x}^T \mathbf{A}_i \mathbf{x} + \mathbf{b}_i^T \mathbf{x}$ . Output  $(\mathbf{A}_i, \mathbf{b}_i, y_i)_{i \leq m}$ .
- (Goal) Given  $(\mathbf{A}_i, \mathbf{b}_i, y_i)_{i \leq m}$ , find  $\mathbf{x} \in \mathbb{F}_p^n$  such that  $\mathbf{x}^T \mathbf{A}_i \mathbf{x} + \mathbf{b}_i^T \mathbf{x} = y_i$  for all  $i \in [m]$ .

While recent attacks [Beu21, Beu22] have reduced the security of well-known signatures based on the MQ problem as Rainbow, in our work, we set the parameter set by executing the estimator of [BMSV22, EVZB23] to estimate the hardness of given an instance of MQ against all existing attack algorithms.

## 4 Multi-instance Vector Commitment

We present our new construction of vector commitment from multi-instance PPRF in Figure 1, its security has relied on the random oracle and the randomness property of multi-instance PPRF. We prove the multi-hiding and extractable-binding properties in Theorem 10 and Theorem 11 respectively.

### 4.1 Multi-instance Vector Commitment Scheme

**Multi-instance PPRF.** We make the use of multi-instance PPRF instantiation from [BCC<sup>+</sup>24], in this construction, the  $(N, \tau)$ -PPRFs is constructed from “Davies-Meyer” construction of a length-doubling PRG:  $\text{PRG}(x) \rightarrow (\pi_0(x) \oplus x, \pi_1(x) \oplus x)$ , where  $(\pi_0, \pi_1)$  are pseudorandom permutations over  $\{0, 1\}^\lambda$  that are instantiated by using two different keys  $(\text{salt}_0, \text{salt}_1)$ . This PRG construction is a  $(N, \tau)$ -instance  $(t, \epsilon)$ -secure PRG (Definition 3).

The construction of  $(N, \tau)$ -PPRFs of length  $2^D$  is described in a recursively tree below:

- Samples a seed ( $\text{seed}$ )  $\leftarrow_r \{0, 1\}^\lambda$  as the root of tree and two salts  $(\text{salt}_0, \text{salt}_1) \in \{0, 1\}^s$  used to define the PRG.
- For each  $i \in [2^j], j \in [D]$ , given a node  $\text{seed}_i$ , the two children of next layer  $(\text{seed}_{2i}, \text{seed}_{2i+1})$  are defined in the following way:

$$(\pi_0(\text{seed}_i) \oplus \text{seed}_i, \pi_1(\text{seed}_i) \oplus \text{seed}_i).$$

- Denote  $\text{PPRF}_{\text{salt}}(\text{seed}, i)$  as the value of  $i$ -th leaf and  $\text{CoPath}_{\text{salt}}(\text{seed}, i)$  as the set of intermediate nodes that can be used to recover all leaves except the  $i$ -th one.

We note that, for repetition, the generation of tree uses the same salts  $(\text{salt}_0, \text{salt}_1)$  (random permutations) while the master key  $\text{seed}$  for each tree is sampled randomly to avoid collision between node values.

**Construction.** Our vector commitment, denoted as  $(N, \tau, 2^D) - \text{VC}$ , is illustrated in Figure 1. Essentially, this scheme is based on  $\tau$  binary-tree structures with  $2^D$  leaves where verifier knows all-but- $\tau$  leaves, multi-instance  $(N, \tau, 2^D) - \text{VC}$  needs to satisfy two properties: *hiding* and *binding*.

- Hiding means that verifier learns nothing about the  $\tau$  unopened leaves, keeping it completely confidential.
- Binding ensures that prover must convince verifier that they know the values of the hidden leaves, with only a negligible chance of cheating after committing.

These properties hold up under a security model involving multiple game repetitions, where the  $\mathcal{A}$  is given the information related to  $N$  sampled trees in each of the  $\tau$  repetitions. The parameters of the multi-instance  $(N, \tau, 2^D) - \text{VC}$  are directly aligned with those in the  $(N, \tau)$ -PPRFs.

Our vector commitment is expanded using PPRF from  $\tau$  random seeds with the same salt value throughout all trees. The last leaves then are used as an input of a random oracle to compute  $(\text{sd}_j^i, \text{com}_j^i)_{i \in [0, \tau], j \in [0, 2^D]}$ , the usage of random oracle is unavoidable for security proof since it 1) makes sure the adversary in hiding game learns nothing about the secret leaves even this adversary can query more than one query, and 2) allows us to define the bit-length of commitment for each  $\text{sd}$  of being large enough, therefore, the extractor in the binding game can not find the collision with high probability.



## PARAMETERS:

- Tree width  $2^D \in \mathbb{N}$ , computational security parameter  $\lambda$ .
- A repetition parameter  $\tau \in \mathbb{N}$ .
- $\text{PRG} : \{0, 1\}^* \rightarrow \{0, 1\}^{\tau\lambda}, H : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda \times \{0, 1\}^{2\lambda}$  pseudorandom generators and  $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$  collision-resistant hash functions.
- $H, H_1$  are modeled as random oracles.
- An instantiation of multi-instance of PPRF with a fixed salt  $\text{salt} = (\text{salt}_0, \text{salt}_1) \in \{0, 1\}^{2\lambda}$ .

## CONSTRUCTION:

- **Commit**(seed, salt,  $1^\lambda$ ):
  1. Compute  $\text{seed}^0 \parallel \dots \parallel \text{seed}^{\tau-1} \leftarrow \text{PRG}(\text{seed}; \tau\lambda)$ .
  2. For  $i \in [0, \tau)$ :
    - For  $j \in [0, 2^D)$ , compute:
      - (a)  $\text{seed}_j^i \leftarrow \text{PPRF}_{\text{salt}}(\text{seed}^i, j)$ .
      - (b)  $(\text{sd}_j^i, \text{com}_j^i) \leftarrow H(\text{seed}_j^i)$ .
    - Compute  $\text{com}^i \leftarrow H_1(\text{com}_0^i, \dots, \text{com}_{2^D-1}^i)$ .
  3. Compute  $h_{\text{com}} \leftarrow H_1(\text{com}^0 \parallel \dots \parallel \text{com}^{\tau-1})$ . Let  $\text{decom} := (\text{seed}, \text{salt})$ .
  4. Output  $(h_{\text{com}}, \text{decom}, (\text{sd}_j^i)_{i \in [0, \tau), j \in [0, 2^D)})$ .
- **Open**(decom,  $I = \cup_{i=0}^{\tau-1} [0, 2^D) \setminus \{\Delta_i\}$ ):
  1. For  $i \in [0, \tau)$ , compute:
    - (a) Write  $I^i = [0, 2^D) \setminus \{\Delta_i\}$ .
    - (b) Compute  $\text{pdecom}_i \leftarrow (\text{salt}, \text{CoPath}_{\text{salt}}(\text{seed}^i, \Delta_i), \text{com}_{\Delta_i}^i)$ .
  2. Output the all-but- $\tau$  opening  $\text{pdecom}_I := (\text{pdecom}_i)_{i \in [0, \tau)}$ .
- **Verify**( $h_{\text{com}}, I, \text{pdecom}_I$ ):
  1. For  $i \in [0, \tau)$ , compute:
    - (a) Write  $I^i = [0, 2^D) \setminus \{\Delta_i\}$ .
    - (b) Recompute  $(\text{sd}_j^i, \text{com}_j^i)_{j \in I^i}$  from  $\text{pdecom}_i$  using  $\text{PPRF}_{\text{salt}}$ .
    - (c) Compute  $\text{com}^i \leftarrow H_1(\text{com}_0^i \parallel \dots \parallel \text{com}_{2^D-1}^i)$ .
  2. Compute  $h'_{\text{com}} \leftarrow H_1(\text{com}^0 \parallel \dots \parallel \text{com}^{\tau-1})$ .
  3. If  $h'_{\text{com}} \neq h_{\text{com}}$ , output  $\perp$ ; otherwise output  $(\text{sd}_j^i)_{i \in [0, \tau), j \in \{I^i\}}$ .

**Fig. 1.** Construction  $(N, \tau, 2^D) - \text{VC}^{H, H_1}$  of Multi-instance of Vector Commitment

## 4.2 Security Proofs

To fit into signature syntax where the adversary can query  $N$ -times to signing oracle, we prove our  $(N, \tau, 2^D) - \text{VC}^{H, H_1}$  is secure in  $N$ -multi-hiding game that is defined as Definition 7 except adversary making  $N$  queries and each query giving adversary information of all-but- $\tau$  vector commitment.

**Theorem 10 (Multi-hiding).** *Assume that PPRF is a  $(N, \tau)$ -instance  $(t, \epsilon_{\text{PPRF}})$ -secure PPRF, PRG,  $H$  are pseudorandom generator with output domains of  $\tau\lambda, 3\lambda$  bits respectively. Then, for any PPT adversary  $\mathcal{A}$  in the adaptive  $N$ -multi-hiding vector commitment game against the  $(N, \tau, 2^D) - \text{VC}^{H, H_1}$  scheme, we have*

$$\mathcal{A}_{\mathcal{A}, \text{VC}}^{\text{MulHiding}}[N] \leq N \cdot \epsilon_{\text{PRG}^{\tau\lambda}} + \epsilon_{\text{PPRF}} + N \cdot \tau \cdot \epsilon_{\text{PRG}^{3\lambda}}$$

*Sketch proof.* We proceed by gradually replacing all intermediate node values along the hidden path (from the root to the hidden leaves) with random values, leveraging the pseudorandomness of the PRG.

First, we replace the seeds defining the roots of the  $\tau$ -trees with uniformly random values, i.e.,

$$(\text{seed}^0 \parallel \dots \parallel \text{seed}^{\tau-1}) \leftarrow \{0, 1\}^{\tau\lambda}.$$

By the security of PRG, the adversary's advantage in distinguishing this step is at most  $N \cdot \epsilon_{\text{PRG}^{\tau\lambda}}$ .

Next, for each  $i$ -th PPRF tree ( $i \in [0, \tau)$ ), we replace the PRG evaluations along the path from  $\text{seed}^i$  to  $\Delta_i$  with truly random values. Since the PRG used in the PPRF is a  $(N, \tau)$ -instance  $(t, \epsilon_{\text{PRG}})$ -secure PRG, the adversary's advantage after replacing all intermediate nodes along the hidden path is

at most  $\epsilon_{\text{PPRF}}$ . (Here, the factor  $N$  is omitted since the PPRF is an  $N$ -multi-instance secure PPRF.) After this step, all  $(\text{seed}_j^i)_{i \in [0, \tau], j \in [0, 2^D]}$  become indistinguishable from truly random values.

Finally, since  $H$  is modeled as a random oracle, we replace  $(\text{sd}_j^i)_{i \in [0, \tau], j \in [0, 2^D]}$  with uniformly random values. The adversary's advantage in distinguishing this final step is at most  $N \cdot \tau \cdot \epsilon_{\text{PRG}^{3\lambda}}$ . This completes the proof sketch.

*Proof.* We proceed via a sequence of hybrid experiments, where each transition relies on the pseudorandomness of PRGs and the security of the PPRF. We recall that PPRF is a  $(N, \tau)$ -instance  $(t, \epsilon_{\text{PPRF}})$ -secure PPRF, and that PRG,  $H$  are pseudorandom generators with advantages at most  $\epsilon_{\text{PRG}^{\tau\lambda}}, \epsilon_{\text{PRG}^{3\lambda}}$ , respectively.

- **Experiment 0** ( $\text{Exp}^0$ ). In this experiment, the  $\tau$ -trees of each  $k$ -th instance ( $k \in [N]$ ) are generated according to the actual scheme described in Figure 1. Specifically, the roots  $(\text{seed}_j^i)_{i \in [0, \tau]}$  are generated via a pseudorandom generator PRG, and the expansion is performed using  $\text{PPRF}(\text{seed}_j^i)$  to derive the  $2^D$  leaves of each tree. The values  $(\text{sd}_j^i, \text{com}_j^i)_{i \in [0, \tau], j \in [0, 2^D]}$  are then computed using a random oracle  $H$ . The adversary  $\mathcal{A}$  learns all  $\text{sd}_j^i$  except for  $(\text{sd}_{\Delta_i}^i)_{i \in [0, \tau]}$ .
- **Experiment 1** ( $\text{Exp}^1$ ). This experiment is identical to Experiment 0, except that the root seeds  $(\text{seed}_j^i)_{i \in [0, \tau]}$  are sampled uniformly at random from  $\{0, 1\}^{\tau\lambda}$  instead of being generated by PRG. By the security of PRG, which is  $(t, \epsilon_{\text{PRG}^{\tau\lambda}})$ -secure, we obtain the following bound:

$$|\Pr[\text{Exp}^0(\lambda) = 1] - \Pr[\text{Exp}^1(\lambda) = 1]| \leq N \cdot \epsilon_{\text{PRG}^{\tau\lambda}}.$$

- **Experiment 2** ( $\text{Exp}^2$ ). This experiment differs from Experiment 1 in the way the hidden paths  $(\Delta_i)_{i \in [0, \tau]}$  are handled. Specifically, all intermediate nodes along these paths are now sampled uniformly from  $\{0, 1\}^\lambda$  instead of being computed via PPRF. By the security of PPRF as a  $(N, \tau)$ -instance  $(t, \epsilon_{\text{PPRF}})$ -secure PPRF, we obtain the bound:

$$|\Pr[\text{Exp}^1(\lambda) = 1] - \Pr[\text{Exp}^2(\lambda) = 1]| \leq \epsilon_{\text{PPRF}}.$$

- **Experiment 3** ( $\text{Exp}^3$ ). In this final experiment, all nodes on the co-paths to  $(\Delta_i)_{i \in [0, \tau]}$ , as well as the leaves  $(\text{seed}_{\Delta_i}^i)_{i \in [0, \tau]}$ , are sampled uniformly at random. This differs from Experiment 2 in that the values  $(\text{sd}_{\Delta_i}^i)_{i \in [0, \tau]}$  are now chosen randomly rather than being generated using  $H$ . Since the number of such values that need to be replaced is  $N \cdot \tau$ , and since PRG is  $(t, \epsilon_{\text{PRG}^{3\lambda}})$ -secure, we obtain:

$$|\Pr[\text{Exp}^2(\lambda) = 1] - \Pr[\text{Exp}^3(\lambda) = 1]| \leq N \cdot \tau \cdot \epsilon_{\text{PRG}^{3\lambda}}.$$

Applying the triangle inequality over all the hybrid transitions, we conclude that:

$$|\Pr[\text{Exp}^0(\lambda) = 1] - \Pr[\text{Exp}^3(\lambda) = 1]| \leq N \cdot \epsilon_{\text{PRG}^{\tau\lambda}} + \epsilon_{\text{PPRF}} + N \cdot \tau \cdot \epsilon_{\text{PRG}^{3\lambda}},$$

which completes the proof.  $\square$

**Theorem 11 (Extractable-binding).** *Let  $H, H_1$  be random oracles, for any PPT adversary  $\mathcal{A}^{H, H_1}$  making  $|Q|$  queries to  $H$  and  $|Q_1|$  queries to  $H_1$ , there exists an extractor  $\text{Ext}$  such that*

$$\text{Adv}_{\mathcal{A}, \text{VC}}^{\text{ExtBinding}} \leq \frac{(|Q_1| + \tau + 1)^2 + (|Q| + \tau \cdot 2^D)^2}{2^{2\lambda}}$$

*Sketch proof.* We construct a straight-line extractor by analyzing the queries made by  $\mathcal{A}$  to the two random oracles  $H$  and  $H_1$ . Specifically, the extractor observes the complete list of queries submitted by  $\mathcal{A}$  to these oracles. The total number of queries that  $\mathcal{A}$  can make to the oracle  $H$  is at most  $|Q| + \tau \cdot 2^D$ , while the total number of queries to  $H_1$  is at most  $|Q_1| + \tau + 1$ .

*Proof.* We construct an extractor  $\text{Ext}$  by leveraging the list of queries made by  $\mathcal{A}$  to the random oracles  $H$  and  $H_1$ . The extractor proceeds as follows:

1. The extractor  $\text{Ext}$  searches for the pre-image  $\text{com}^0 \| \dots \| \text{com}^{\tau-1}$  of  $h_{\text{com}}$  in the query list  $Q_1$ . If no such pre-image exists or if it is multiply defined,  $\text{Ext}$  outputs  $\perp$  for all messages. The adversary  $\mathcal{A}$  succeeds if it can find at least one valid pre-image or a collision for  $H$  within  $Q$ . The probability of this event occurring is bounded by  $|Q_1|^2 / 2^{2\lambda}$ .

2. For each  $i \in [0, \tau)$ , the extractor  $\text{Ext}$  attempts to find the pre-image  $\text{com}_0^i \parallel \dots \parallel \text{com}_{2^D-1}^i$  of  $\text{com}^i$ . If no valid pre-image is found or if it is multiply defined,  $\text{Ext}$  outputs  $\perp$  for all messages from the  $i$ -th tree. The probability that  $\mathcal{A}$  succeeds for any given  $\text{com}^i$  is bounded by  $\tau \cdot |Q_1|^2 / 2^{2\lambda}$ .
3. For each  $\text{com}_j^i$  where  $i \in [0, \tau), j \in [0, 2^D)$ , the extractor  $\text{Ext}$  utilizes the list of queries in  $Q$  to extract the corresponding tree leaf  $\text{seed}_j^i$ . The extractor defines  $\text{sd}_j^i$  as  $(\text{sd}_j^i, \text{com}_j^i) = H(\text{seed}_j^i)$ . If no valid  $\text{seed}_j^i$  is found,  $\text{Ext}$  sets  $\text{sd}_j^i = \perp$ . The probability that  $\mathcal{A}$  succeeds at this step is bounded by  $(|Q| + \tau \cdot 2^D)^2 / 2^{2\lambda}$ .

We now provide additional details regarding the final step. For each  $\text{com}_j^i$  where  $i \in [0, \tau), j \in [0, 2^D)$ ,  $\text{Ext}$  uses the query list  $Q$  to extract the corresponding tree leaf  $\text{seed}_j^i$ . The bit length of  $\text{com}_j^i$  is  $2\lambda$  bits.

- If a unique  $\text{seed}_j^i$  exists, the extractor defines  $\text{sd}_j^i$  as  $(\text{sd}_j^i, \text{com}_j^i) = H(\text{seed}_j^i)$ .
- If no pre-image  $\text{seed}_j^i$  is found in the query list  $Q$ , or if multiple distinct values  $\text{seed}_j^i$  map to the same output  $(\text{sd}_j^i, \text{com}_j^i)$ , then  $\text{Ext}$  sets  $\text{sd}_j^i = \perp$ , indicating failure to extract a unique witness. The event in which at least one  $\text{com}_j^i$  has multiple distinct pre-images directly contradicts the collision-resistance property of  $H$ . Therefore, the probability of this event occurring is at most  $(|Q| + \tau \cdot 2^D)^2 / 2^{2\lambda}$ .

□

## 5 Shorter VOLEitH-based Signature

### 5.1 Overview of Signature

We introduce a signature scheme based on the MQ problem from the VOLEitH paradigm, where the hardness assumption is the MQ problem. The signature is built on a zero-knowledge proof that is publicly verifiable and is constructed from a designated-verifier ZK protocol by combining the VOLE-in-the-Head technique [BBD<sup>+</sup>23], SoftspokenOT [Roy22] and our multi-instance VC. The designated-verifier ZK is an efficient proof of knowledge of witness for  $\text{MQ}_{p,m,n}$  problem using QuickSilver [YSWW21]. Finally, the Fiat-Shamir transform is used to compile the publicly verifiable zero-knowledge protocol into an MQ-based signature scheme.

Given a VOLE correlation where prover holds two vectors  $\mathbf{U}, \mathbf{V} \in \mathbb{F}_2^\ell \times \mathbb{F}_{2^\lambda}^\ell$  and verifier holds a value  $\Delta$  and a vector  $\mathbf{W}$ ,  $(\Delta, \mathbf{W}) \in \mathbb{F}_{2^\lambda} \times \mathbb{F}_{2^\lambda}^\ell$  such that  $\mathbf{W} := \Delta \cdot \mathbf{U} + \mathbf{V}$ . The VOLEitH-based zero-knowledge proof is constructed from the fact that in VOLE-zero-knowledge proof the value  $\Delta$  held by verifier can be revealed after prover has committed to the witness, without compromising security. To achieve this, SoftspokenOT is employed to establish a VOLE correlation where  $\Delta$  can be selected and disclosed after the commitment phase. Once  $\Delta$  is revealed, verifier learns  $\mathbf{W}$  and can verify the validity of the committed values, leading to the construction of a public-coin zero-knowledge protocol. Below, we list the main ingredients within the VOLE-in-the-Head paradigm (for details, we refer the reader to the FAEST specs):

- Prover firstly creates random multi-instance all-but-one vector commitment  $\text{VC.Commit}(\text{seed}, \text{salt}, 2^D, \tau) \rightarrow (h_{\text{com}}, (\text{sd}_j^i)_{i \in [0, \tau), j \in [0, 2^D)})$ , expands to small VOLEs using PRGs, and then combines  $\tau$  of them to a big VOLE that satisfies the desired security and later is used for the underlying ZK proof. Prover holds  $\mathbf{U}, \mathbf{V} \in \mathbb{F}_2^\ell \times \mathbb{F}_{2^\lambda}^\ell$ .
- Prover sends to verifier  $(h_{\text{com}}, \mathbf{C})$  where  $\mathbf{C} \in \mathbb{F}_2^{\ell \times (\tau-1)}$ , i.e.,  $\mathbf{C}$  is used to derandomize  $\tau$  different VOLEs, to get a unique vector  $\mathbf{U} \in \mathbb{F}_2^\ell$  across all instances.
- $\mathbf{V}$  samples  $H^{\text{UHF}} \in \mathbb{F}_2^{(\lambda+16) \times \ell}$  a universal hash function and sends it to  $\mathbf{P}$ .
- Prover sends the VOLE consistency proof  $\tilde{\mathbf{u}}, \tilde{\mathbf{V}}$  to verifier, which is based on SoftspokenOT using the linear universal hash function. Given  $\tilde{\mathbf{u}} = H^{\text{UHF}} \cdot \mathbf{U}$ ,  $\tilde{\mathbf{V}} = H^{\text{UHF}} \cdot \mathbf{V}$ . Note that instead of sending  $\tilde{\mathbf{V}}$  directly, the signer can send a collision-resistant hash of this to save some communication. The verifier can still check by recomputing  $\tilde{\mathbf{V}}$  (from  $\Delta$ ,  $\tilde{\mathbf{u}}$ ,  $\mathbf{W}$ ).
- Verifier samples random challenge and sends it to prover. This challenge is used for VOLE-based zero-knowledge proof instantiated by QuickSilver [YSWW21].
- Prover sends QuickSilver proof to verifier. The QuickSilver proof part  $\gamma \in \mathbb{F}_p^n$  (to hide witness) and  $(\text{QS}_0, \text{QS}_1) \in \mathbb{F}_{2^{128}}$ .

- After prover sends all proofs, verifier opens the value  $\Delta$  of VOLE correlation.
- From  $\Delta$ , prover opens all-but-one vector commitment by sending the opening of VC to verifier, i.e.,  $\text{pdecom}_I = \{\text{CoPath}_{\text{salt}}(\Delta_i, \text{seed}^i), \text{com}_{\Delta_i}^i\}_{i < \tau}$  for each of the  $\tau$ -PPRF instances, opening all positions except  $\Delta = (\Delta_0, \dots, \Delta_{\tau-1}) \in \mathbb{F}_{2^\lambda}$  (bits concatenation), after that verifier reconstructs  $\mathbf{W}$ , checks the VOLE consistency and QuickSilver proof.

## 5.2 Parameter Choices and Signature Size

In this section, we explain how to select parameters for our new signature scheme with a security level of  $\lambda$ .

- The field size of  $\mathbb{F}_p$  depends on the security of  $\text{MQ}_{p,m,n}$  problem to ensure the security level of  $\lambda$  bits. For feasible implementation,  $\mathbb{F}_p$  is chosen as an extension field of  $\mathbb{F}_2$  and subfield of  $\mathbb{F}_{2^\lambda}$ . We run the estimator from [BMSV22, EVZB23] to estimate the running time and memory usages of the existing attack algorithms for the MQ problem, where we choose the number of unknowns and the number of equations being equal ( $m = n$ ) and other parameters for each algorithm is chosen default by the estimator. For the memory representation, following from [Wan22], observe that when choosing  $p, n$  for a given security level, if  $n$  decreases as  $q$  increases, but  $n \log p$  will slightly increase. Since the asymptotic signature size is proportional to  $n \log p$ , the signature size also increases if  $p$  increases. However, a larger  $p$  dramatically reduces  $n$  in the multivariate quadratic equations, so the memory storage is also reduced. We choose the parameter choice for  $p = 2$ ,  $m = n = 150$  and  $p = 4, m = n = 88$ .
- The repetitions  $\tau \in \mathbb{N}$ , the speed of the signing algorithm is dominated by establishing  $\tau$  vector commitments of length  $N = 2^D$ . The choice for  $\tau$  offers tradeoffs between signature size and running time; a small  $\tau$  means a smaller signature size and vice.
- SoftspokenOT parameters, we use a sVOLE constructed from SoftspokenOT of length  $\ell = n \log p + \lambda + (\lambda + 16)$  where first  $n \log p$ -sVOLE correlations are used to hide the witness of length  $n$ , next  $\lambda$ -sVOLE correlations are for nullity check for polynomial set using Quicksilver and last  $(\lambda + 16)$ -correlations is added more to make sure verifier learns nothing about  $\mathbf{U}$  in VOLE consistent checks.

**Public key size.** Our public key size includes a seed  $\text{sd} \in \{0, 1\}^\lambda$  that is used to get MQ instance  $(\mathbf{A}_i, \mathbf{b}_i)_{i \leq n}$  by using PRG and  $(y_i)_{i \leq n} \in \mathbb{F}_p^n$  as  $y_i := \mathbf{x}^T \mathbf{A}_i \mathbf{x} + \mathbf{b}_i^T \mathbf{x}$  where  $\mathbf{x}$  is hidden secret in  $\text{MQ}_{p,n,n}$ . This leads to a public key size of  $(\lambda + n \log p)$  bits.

**Signature size.** The signer generates the signature  $\sigma$  which consists of:

- The key  $\text{salt} \in \{0, 1\}^{2\lambda}$  for multi-instances PPRF.
- 2 hash values  $h_1, h_2 \in \{0, 1\}^{2\lambda}$  (used to generate challenges in Fiat-Shamir transformation).
- The vector  $\mathbf{C} \in \mathbb{F}_2^{\ell \times (\tau-1)}$ .
- The hashed VOLE secrets  $\tilde{\mathbf{u}}, \tilde{\mathbf{V}}$ .
- The QuickSilver proof part  $\gamma \in \mathbb{F}_p^n$  (to hide witness) and  $(\text{QS}_0, \text{QS}_1) \in \mathbb{F}_{2^{128}}$ .
- The partial  $\text{pdecom}_I = \{\text{CoPath}_{\text{salt}}(\Delta_i, \text{seed}^i), \text{com}_{\Delta_i}^i\}_{i < \tau}$  for each of the  $\tau$ -PPRF instances, opening all positions except  $\Delta = (\Delta_0, \dots, \Delta_{\tau-1}) \in \mathbb{F}_{2^{128}}$ .

The signature size is

$$\underbrace{(\tau - 1) \cdot (n \cdot \log p + 2\lambda + 16)}_{\mathbf{C}} + \underbrace{(\lambda + 16)}_{\tilde{\mathbf{u}}} + \underbrace{n \cdot \log p}_{\gamma} + \underbrace{2\lambda}_{\text{QS}_0, \text{QS}_1} + \underbrace{2 \cdot \lambda}_{\tilde{\mathbf{V}}} + \underbrace{2 \cdot \lambda}_{\text{salt}} + \underbrace{4 \cdot \lambda}_{h_1, h_2} + \underbrace{\tau \cdot D \cdot \lambda}_{\text{CoPath}} + \underbrace{2 \cdot \tau \cdot \lambda}_{\text{com}_{\Delta_i}^i}$$

We report in Table 1 the estimated sizes of our signatures, computed using a Python script, for the corresponding parameter sets. For small field sizes, the signatures range from 3.1 KB to 3.6 KB. We also provide a comparison with other MQ-based signature schemes in terms of signature size only, as we do not include efficiency comparisons due to the lack of implementation. We choose the field is extension fields of  $\mathbb{F}_2$  and has small size, allowing operations to be implemented with XOR and AND. This choice reduces the computational cost associated with polynomial operations in the underlying MQ assumption.

**Table 1.** Comparison of our new signature scheme with other MQ-based signatures in terms of parameter choice, estimated security, sizes of public key, and signature for the security level I of NIST’s security standard. Each signature has two variants (**fast** and **short**),  $N$  is the number of parties in MPCitH-based signature,  $D$  is the depth of GGM tree-based VC in VOLEitH-based signature,  $\tau$  is the number of repetitions.

| Scheme                                          | MQ parameters  |         | MPC/VOLE parameters |        | Size       |           |
|-------------------------------------------------|----------------|---------|---------------------|--------|------------|-----------|
|                                                 | $p$            | $m = n$ | $N/D$               | $\tau$ | Public key | Signature |
| MQ-DSS [CHR <sup>+</sup> 16]                    | 31             | 48      | -                   | -      | 46 B       | 28400 B   |
| MudFish [Beu20]                                 | 4              | 88      | 4                   | 68     | 38 B       | 14640 B   |
| Mesquite <sub>fast</sub> [Wan22]                | 4              | 88      | 8                   | 49     | 38 B       | 9492 B    |
| Mesquite <sub>short</sub>                       | 4              | 88      | 32                  | 28     | 38 B       | 8844 B    |
| Fen22 <sub>fast</sub> [Fen22]                   | 251            | 40      | 32                  | 40     | 56 B       | 8488 B    |
| Fen22 <sub>short</sub>                          | 251            | 40      | 256                 | 25     | 56 B       | 7114 B    |
| MQOM <sub>fast</sub> [BFR23]                    | 251            | 43      | 32                  | 34     | 59 B       | 7809 B    |
| MQOM <sub>short</sub>                           | 251            | 43      | 256                 | 22     | 59 B       | 6575 B    |
| MQOM <sub>fast</sub>                            | 31             | 49      | 32                  | 35     | 47 B       | 7621 B    |
| MQOM <sub>short</sub>                           | 31             | 49      | 256                 | 20     | 47 B       | 6348 B    |
| Our <sub>short</sub>                            | 2              | 150     | 12                  | 11     | 35 B       | 3188 B    |
|                                                 | 4              | 88      | 12                  | 11     | 38 B       | 3224 B    |
| Our <sub>fast</sub>                             | 2              | 150     | 8                   | 16     | 35 B       | 3549 B    |
|                                                 | 4              | 88      | 8                   | 16     | 38 B       | 3600 B    |
| KuMQuat <sub>short</sub> [BBM <sup>+</sup> 24a] | 2              | 152     | 11-12               | 11     | 35 B       | 2555 B    |
|                                                 | 2 <sup>8</sup> | 48      | 11-12               | 11     | 64 B       | 2890 B    |
| KuMQuat <sub>fast</sub>                         | 2              | 152     | 8                   | 16     | 35 B       | 3028 B    |
|                                                 | 2 <sup>8</sup> | 48      | 8                   | 16     | 64 B       | 3588 B    |

## 6 Application to Post-quantum Signatures

### 6.1 MPCitH-based signatures from Vector Commitment

As mentioned in Section 2, our vector commitment can be plugged into the MPCitH-based signatures that satisfy:

- Use a PRG to get a commitment for the witness shares from the leaves of the PPRF-tree.
- And to get provable security, this PRG needs to be modeled as a random oracle.

The adaption is done by adjusting the output of random oracle  $H$  in our vector commitment to fit with the output domain of PRG used in MPCitH-based signatures. We show below how to adapt it into the MPCitH-based signature of [BCC<sup>+</sup>24]. Since we take advantage of their multi-instance PPRF, we do not claim efficiency, we contribute to a shorter and clearer proof security while for others MPCitH-based signatures [FJR22, AGH<sup>+</sup>23, CCJ23, BCC<sup>+</sup>24], we achieve straightforward concrete efficiency as shown in [BCC<sup>+</sup>24] because of the efficiency of multi-instance PPRF.

In the signing algorithm of [BCC<sup>+</sup>24] (Figure 5, phase 1), after executing the PPRF to get  $\text{seed}_i^e \leftarrow \text{PPRF}_{\text{salt}}(\text{seed}^e, i)$  for all  $i \in [2^D]$ ,  $e \in [\tau]$ ; signer applies a PRG on each  $\text{seed}_i^e$  to compute the commitment and the randomness  $(x_i^e, r_i^e, u_i^e)$  that are used to mask the witness shares and the material of the underlying identification scheme respectively,  $(x_i^e, r_i^e, u_i^e, \text{com}_i^e) \leftarrow \text{PRG}(\text{seed}_i^e)$  where  $\text{com}_i^e \in \{0, 1\}^\lambda$ . Following the proof soundness of the identification scheme, this PRG needs to be binding (lemma 23 of [BCC<sup>+</sup>24]), therefore, we substitute this PRG and PPRF construction by our vector commitment where we set  $H(\text{seed}_i^e) \rightarrow (\text{sd}_i^e, \text{com}_i^e)$  and  $\text{sd}_i^e := (x_i^e, r_i^e, u_i^e)$  and  $\text{com}_i^e \in \{0, 1\}^\lambda$ . As a result, in the advantage of  $\mathcal{A}$  in unforgeability game (Theorem 17 of [BCC<sup>+</sup>24]), the term  $\epsilon_{\text{PPRF}} + \epsilon_{\text{PRG}}$  can be simplified by  $\mathcal{A}_{\mathcal{A}, \text{VC}}^{\text{MulHiding}}[q_s]$  ( $q_s$  is number of queries that  $\mathcal{A}$  made to the signing oracle).

### 6.2 Faster VOLEitH-based signatures

Our vector commitment can be a direct replacement for the vector commitment in VOLEitH-based signature schemes. We show the concrete efficiency of our vector commitment by comparing it with the vector commitment in FAEST signature for 128 bits of security below.

We use the instantiation of [BCC<sup>+</sup>24] to instantiate multi-instance PPRF using only fixed-key block cipher. For security level  $\lambda = 128$  bits, taking advantage of AES-NI, [BCC<sup>+</sup>24] shows that their construction has 14 – 18% performance improvement compared to others hash-based PPRF used in existing MPCitH-based signatures such as [AGH<sup>+</sup>23], as the methodology under the hood of using PPRF in VOLEitH and MPCitH are similar, multi-instance PPRF has a similar performance when dropping into VOLEitH (see Section 6.3 for details).

In [BCC<sup>+</sup>24], the expansion of the PPRF-tree is constructed by  $\text{PRG} : x \rightarrow (\text{AES}_{K_0}(x) \oplus x, \text{AES}_{K_1}(x) \oplus x)$ , where  $x$  is the value of parent node and the left child and right child are defined by  $\text{AES}_{K_0}(x) \oplus x$  and  $\text{AES}_{K_1}(x) \oplus x$ , respectively. The key  $K = (K_0, K_1)$  is fixed for all  $\tau$ -PPRF instances.

In total, the computation cost to establish our vector commitment (VC.Commit) includes:

- Expanding PPRF-tree:  $(1/2 \cdot N \cdot \tau)$  calls to  $(\text{AES}_{K_0}, \text{AES}_{K_1})$  which is instantiated by AES-NI, for concrete parameters of signatures where  $N \cdot \tau$  ranging from  $2^{11}$  to  $2^{13}$ , which is efficient since encrypting using AES-NI only takes 1.3 cycles per byte [MSY21].
- Obtaining commitment:  $N \cdot \tau$  calls to a hash function  $H$  with output  $3\lambda$  and  $\tau$  calls to a hash function  $H_1$  with output  $2\lambda$ . These hash functions can be instantiated by SHAKE128 as in [BBD<sup>+</sup>23]. Using SHAKE to get commitments is unavoidable since the adversary in the forgery attack can query polynomial times to vector commitment, leading to the output domain of commitment being at least  $2\lambda$  for a security level  $\lambda$ .

To compare with the vector commitment in FAEST signature, since both use SHAKE to get the commitment, we focus on discussing the efficiency of tree expansion. As per the FAEST specifications, the  $\text{PRG}(K, iv)$  function is implemented using AES in counter mode, where the seed  $K$  acts as the key and the initialization vector  $iv$  is used to initialize a 128-bit counter, which is incremented by one for each subsequent block. The initialization vector  $iv$  is randomly generated for each signature. Specifically,  $\text{PRG}(K, iv) \rightarrow (\text{AES}_K(iv), \text{AES}_K(iv \oplus 1))$ , where  $K$  serves as the AES key and  $iv$  is used as the initial counter value. Our performance improvement comes from using fixed-key AES instead of using AES counter mode with a new key on every call.

### 6.3 Implementation and Evaluation

We follow the reference implementation of FAEST<sup>2</sup> to implement FAEST using our vector commitment. FAEST signature is the first one using the VOLEitH framework to get an efficient signature where the underlying VOLE-based zero-knowledge proof is instantiated by QuickSilver [YSWW21]. We note that this implementation does not include the MQ-based version.

**Table 2.** Performance comparison between our VC variant and the standard FAEST signature scheme using the optimized implementation.

| Variant | Sign ( $\mu s$ ) |       |                                   |      |                                     |       | Verify ( $\mu s$ ) |       |                                   |      |
|---------|------------------|-------|-----------------------------------|------|-------------------------------------|-------|--------------------|-------|-----------------------------------|------|
|         | FAEST            | Our   | Speedup<br>[BBM <sup>+</sup> 24b] |      | Sig. size<br>[BBM <sup>+</sup> 24b] |       | FAEST              | Our   | Speedup<br>[BBM <sup>+</sup> 24b] |      |
| 128S    | 4687             | 3801  | 1.23                              | 1.34 | 5006                                | 4594  | 4600               | 3753  | 1.23                              | 0.99 |
| 128F    | 739              | 633   | 1.17                              | 0.93 | 6336                                | 6052  | 683                | 600   | 1.14                              | 0.78 |
| 256S    | 15446            | 13481 | 1.15                              | 1.22 | 22100                               | 21752 | 15104              | 13116 | 1.15                              | 0.58 |
| 256F    | 3271             | 2978  | 1.10                              | 1.13 | 28400                               | 28084 | 3037               | 2797  | 1.09                              | 0.57 |
| EM-128S | 4504             | 3516  | 1.28                              | 1.38 | 4566                                | 4170  | 4437               | 3485  | 1.27                              | 1.01 |
| EM-128F | 638              | 538   | 1.19                              | 1.06 | 5696                                | 5444  | 637                | 547   | 1.16                              | 0.78 |
| EM-256S | 14702            | 12580 | 1.17                              | 1.20 | 20956                               | 19744 | 14465              | 12464 | 1.16                              | 0.62 |
| EM-256F | 2841             | 2618  | 1.09                              | 1.02 | 26736                               | 26036 | 2804               | 2580  | 1.09                              | 0.54 |

Our implementation uses eXtended Keccak Code Package (XKCP) for SHA3 and OpenSSL libraries for fixed-key AES. Note that XKCP and OpenSSL are already optimized for the x86 archi-

<sup>2</sup> <https://github.com/faest-sign/faest-ref>

ture with the AVX2 and AES-NI instruction set extensions. All experiments are performed on an AMD Ryzen 5 5600X 6-Core Processor running GNU/Linux using a single thread.

We present the performance benchmark for 128-bit and 256-bit security levels in Table 2, using the same parameter set as in FAEST. Our experimental results demonstrate a speedup of 10% to 27% over the original FAEST signature. Notably, our vector commitment performs optimally at 128-bit security, benefiting from the significant hardware acceleration provided by AES-NI. Our signature size remains consistent with FAEST, as our vector commitment serves as a direct replacement for the one in FAEST, offering the same security guarantees.

Additionally, we compare our results with the recent optimization of FAEST, FAESTER [BBM<sup>+</sup>24b], which introduces a technique to batch all  $\tau$ -single tree vector commitments into a big tree, reducing the number of opening values compared to the original vector commitment. As a result, the signature size in FAESTER is smaller than both FAEST and our multi-instance VC-based FAEST. We extract the speedup factor of FAESTER from [BBM<sup>+</sup>24b] (see Table 2), noting that while FAESTER achieves faster signing times, its verification times are not well-performed. In contrast, our approach improves both signing and verification times.

## Acknowledgement

We would like to thank anonymous reviewers, Geoffroy Couteau (IRIF), Kelong Cong (Zama), and Lawrence Roy (Aarhus University) as well as the Faest team (faest.info) for their advice and insightful comments. This work is supported by DIM Math Innovation 2021 (N<sup>o</sup>IRIS: 21003816) from the Paris Mathematical Sciences Foundation (FSMP) funded by the Paris Ile-de-France Region.

## References

- AGH<sup>+</sup>23. C. Aguilar Melchor, N. Gama, J. Howe, A. Hülsing, D. Joseph, and D. Yue. The return of the SDitH. In *EUROCRYPT 2023, Part V, LNCS 14008*, pages 564–596. Springer, Heidelberg, April 2023.
- BBD<sup>+</sup>23. C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, and P. Scholl. Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head. In *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V, Lecture Notes in Computer Science 14085*, pages 581–615. Springer, 2023.
- BBM<sup>+</sup>24a. C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. One tree to rule them all: Optimizing ggm trees and owfs for post-quantum signatures. Cryptology ePrint Archive, Paper 2024/490, 2024. <https://eprint.iacr.org/2024/490>.
- BBM<sup>+</sup>24b. C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. One tree to rule them all: Optimizing ggm trees and owfs for post-quantum signatures. In *Advances in Cryptology – ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings, Part I*, page 463–493, Berlin, Heidelberg, 2024. Springer-Verlag.
- BCC<sup>+</sup>23. D. Bui, H. Chu, G. Couteau, X. Wang, C. Weng, K. Yang, and Y. Yu. An efficient ZK compiler from SIMD circuits to general circuits. Cryptology ePrint Archive, Paper 2023/1610, 2023. <https://eprint.iacr.org/2023/1610>.
- BCC<sup>+</sup>24. D. Bui, E. Carozza, G. Couteau, D. Goudarzi, and A. Joux. Faster signatures from MPC-in-the-head. Cryptology ePrint Archive, Paper 2024/252, 2024. <https://eprint.iacr.org/2024/252>.
- BCdSG24. D. Bui, K. Cong, and C. D. de Saint Guilhem. Improved all-but-one vector commitment with applications to post-quantum signatures. Cryptology ePrint Archive, Paper 2024/097, 2024. <https://eprint.iacr.org/2024/097>.
- BCG<sup>+</sup>19. E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, and P. Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In *CRYPTO 2019, Part III, LNCS 11694*, pages 489–518. Springer, Heidelberg, August 2019.
- Beu20. W. Beullens. Sigma protocols for mq, plk and sis, and fishy signature schemes. In *39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Lecture Notes in Computer Science 12105*. Springer, 2020.
- Beu21. W. Beullens. Improved cryptanalysis of uov and rainbow. page 348–373, Berlin, Heidelberg, 2021. Springer-Verlag.
- Beu22. W. Beullens. Breaking rainbow takes a weekend on a laptop. Cryptology ePrint Archive, Paper 2022/214, 2022. <https://eprint.iacr.org/2022/214>.

- BFR23. R. Benadjila, T. Feneuil, and M. Rivain. Mq on my mind: Post-quantum signatures from the non-structured multivariate quadratic problem. Cryptology ePrint Archive, Paper 2023/1719, 2023. <https://eprint.iacr.org/2023/1719>.
- BGI14. E. Boyle, S. Goldwasser, and I. Ivan. Functional signatures and pseudorandom functions. In *PKC 2014, LNCS* 8383, pages 501–519. Springer, Heidelberg, March 2014.
- BGI16. E. Boyle, N. Gilboa, and Y. Ishai. Function secret sharing: Improvements and extensions. In *ACM CCS 2016*, pages 1292–1303. ACM Press, October 2016.
- BMRS21. C. Baum, A. J. Malozemoff, M. B. Rosen, and P. Scholl. Mac’n’cheese: Zero-knowledge proofs for boolean and arithmetic circuits with nested disjunctions. In *CRYPTO 2021, Part IV, LNCS* 12828, pages 92–122, Virtual Event, August 2021. Springer, Heidelberg.
- BMSV22. E. Bellini, R. H. Makarim, C. Sanna, and J. A. Verbel. An estimator for the hardness of the MQ problem. In *AFRICACRYPT 22, LNCS* 2022, pages 323–347. Springer Nature, July 2022.
- Bui25. D. Bui. Efficient multi-instance vector commitment and application to post-quantum signatures. In *Information Security and Privacy - 30th Australasian Conference, ACISP 2025, Wollongong, NSW, Australia, July 14-16, 2025, Proceedings, Part II, Lecture Notes in Computer Science* 15659, pages 23–41. Springer, 2025.
- BW13. D. Boneh and B. Waters. Constrained pseudorandom functions and their applications. In *ASIACRYPT 2013, Part II, LNCS* 8270, pages 280–300. Springer, Heidelberg, December 2013.
- CCJ23. E. Carozza, G. Couteau, and A. Joux. Short signatures from regular syndrome decoding in the head. In *EUROCRYPT 2023, Part V, LNCS* 14008, pages 532–563. Springer, Heidelberg, April 2023.
- CHR<sup>+</sup>16. M.-S. Chen, A. Hülsing, J. Rijneveld, S. Samardjiska, and P. Schwabe. From 5-pass MQ-based identification to MQ-based signatures. In *ASIACRYPT 2016, Part II, LNCS* 10032, pages 135–165. Springer, Heidelberg, December 2016.
- CLY<sup>+</sup>24. H. Cui, H. Liu, D. Yan, K. Yang, Y. Yu, and K. Zhang. Resolved: Shorter signatures from regular syndrome decoding and vole-in-the-head. Cryptology ePrint Archive, Paper 2024/040, 2024. <https://eprint.iacr.org/2024/040>.
- DS05. J. Ding and D. Schmidt. Rainbow, a new multivariable polynomial signature scheme. In *ACNS 05, LNCS* 3531, pages 164–175. Springer, Heidelberg, June 2005.
- EVZB23. A. Esser, J. Verbel, F. Zveydinger, and E. Bellini. **CryptographicEstimators**: a software library for cryptographic hardness estimation. Cryptology ePrint Archive, Paper 2023/589, 2023. <https://eprint.iacr.org/2023/589>.
- Fen22. T. Feneuil. Building mpcith-based signatures from mq, minrank, rank sd and pkp. Cryptology ePrint Archive, Paper 2022/1512, 2022. <https://eprint.iacr.org/2022/1512>.
- FJR22. T. Feneuil, A. Joux, and M. Rivain. Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In *CRYPTO 2022, Part II, LNCS* 13508, pages 541–572. Springer, Heidelberg, August 2022.
- FS87. A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO’86, LNCS* 263, pages 186–194. Springer, Heidelberg, August 1987.
- GGM84. O. Goldreich, S. Goldwasser, and S. Micali. How to construct random functions (extended abstract). In *25th FOCS*, pages 464–479. IEEE Computer Society Press, October 1984.
- GYW<sup>+</sup>23. X. Guo, K. Yang, X. Wang, W. Zhang, X. Xie, J. Zhang, and Z. Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In *EUROCRYPT 2023, Part I, LNCS* 14004, pages 330–362. Springer, Heidelberg, April 2023.
- IKOS07. Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. Zero-knowledge from secure multiparty computation. In *39th ACM STOC*, pages 21–30. ACM Press, June 2007.
- KPTZ13. A. Kiayias, S. Papadopoulos, N. Triandopoulos, and T. Zacharias. Delegatable pseudorandom functions and applications. In *ACM CCS 2013*, pages 669–684. ACM Press, November 2013.
- MSY21. J.-P. Münch, T. Schneider, and H. Yalame. Vasa: Vector aes instructions for security applications. Cryptology ePrint Archive, Paper 2021/1493, 2021. <https://eprint.iacr.org/2021/1493>.
- Roy22. L. Roy. SoftSpokenOT: Quieter OT extension from small-field silent VOLE in the minicrypt model. In *CRYPTO 2022, Part I, LNCS* 13507, pages 657–687. Springer, Heidelberg, August 2022.
- Sho94. P. W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th FOCS*, pages 124–134. IEEE Computer Society Press, November 1994.
- Wan22. W. Wang. Shorter signatures from mq. Cryptology ePrint Archive, Paper 2022/344, 2022. <https://eprint.iacr.org/2022/344>.
- WYKW21. C. Weng, K. Yang, J. Katz, and X. Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021.
- YSWW21. K. Yang, P. Sarkar, C. Weng, and X. Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *ACM CCS 2021*, pages 2986–3001. ACM Press, November 2021.